

The Live Dashboard

Stream the guarded decision the instant it happens and record it for replay — make the control decision visible and reviewable. The Monogate rule, over WiFi.

ESP32-008 · ~45 MIN · INTERMEDIATE

1. The one idea

Stream the decision, and keep a copy. A **WebSocket** pushes each guarded reading to the browser the moment it happens — not on a refresh — and you log every frame to a replayable trace. The control decision becomes both **visible** (live) and **reviewable** (recorded): Monogate's first rule, over WiFi.

2. Why it matters

Course 7 served a page you refresh; real monitoring *streams*. And a decision you can't replay can't be reviewed — logging each frame turns "it seemed to work" into evidence you can scroll back through. This is the spine the whole bench is built on: a guarded decision that is seen as it happens and recorded for later.

3. Parts & wiring

No new hardware — the ESP32 plus any sensor, running the guard from Course 1 or 4. You're adding software: a WebSocket server and a small trace buffer.

4. Predict (do this before uploading)

- With a WebSocket, who starts each update — the browser asking, or the board pushing?
- A frame is `{time, value, safe, action}`. Roughly how many bytes, and how many per second is plenty?
- WiFi drops for two seconds mid-run. What happens to the live view, and to the recorded trace?

5. Build it — push frames, keep frames

```
#include <WiFi.h>
#include <WebSocketsServer.h>

WebSocketsServer ws(81);          // WebSocket on port 81

struct Frame { uint32_t t; float value, safe; const char* action; };
Frame ring[256]; int head = 0;    // bounded trace buffer – never grows without limit

void publish(float value, float safe, const char* action) {
    ring[head] = { millis(), value, safe, action };
    head = (head + 1) % 256;      // record (oldest frame is overwritten)

    char j[96];
    snprintf(j, sizeof(j), "{\t\t\"t\":%lu,\"v\":%.2f,\"s\":%.2f,\"a\":\"%s\"}",
             millis(), value, safe, action);
    ws.broadcastTXT(j);          // stream to every connected browser
}

void setup() {
    WiFi.begin("YOUR_SSID", "YOUR_PASS");
    while (WiFi.status() != WL_CONNECTED) delay(200);
    Serial.println(WiFi.localIP());
    ws.begin();
}

void loop() {
    ws.loop();
    // ... run the guard each cycle, then:
    // publish(requested, min(requested, LIMIT),
    //         requested > LIMIT ? "clamp" : "pass");
}
```

The browser opens one socket and the board *pushes* a frame every decision — no polling, no refresh. Each frame is also dropped into a fixed-size ring buffer, so the trace is always the last 256 decisions and never eats all the RAM.

6. Test against your prediction

Open the dashboard and watch the value stream as a moving line with no refresh, the `action` flipping to `clamp` the instant the guard fires. Force a clamp and you see it live; then dump the ring buffer over a `/trace` endpoint and replay those same frames — the decision, recorded. Did a WiFi blip drop the live view but leave the trace intact, as you predicted?

7. What goes wrong (pre-loaded debugging)

- **Socket won't connect** → the browser must hit the right port/path (`ws://<ip>:81/`), and the library version matters; confirm with a minimal client first.
- **Choppy or laggy stream** → you're broadcasting too fast or doing heavy work per frame. 10–20 frames/sec of tiny JSON is plenty.
- **The board stutters** → a blocking `delay()` is starving `ws.loop()` . Keep the loop non-blocking, like Course 6 of the robotics track taught.
- **Trace "fills up" / crashes** → an unbounded buffer. Use the fixed ring; bound everything that grows.
- **Frames vanish on a WiFi drop** → expected — the live socket misses them; the ring buffer keeps the record. Reconnect and resume.

8. Checkpoint

The decision is now visible the instant it happens **and** recorded for replay — observability plus evidence. That's the Monogate first rule running on your own board: make the control decision visible, then keep enough of it to review. Every guarded node you build can now be watched live and checked after the fact.

9. Push further

- Match the frame shape to the trace dashboard the bench already uses, so replays drop straight in.
- Mark every `clamp` as a tick on the timeline — a visible record of every time the guard acted.
- Persist the trace to SPIFFS/flash and export it as an evidence packet.